**Министерство науки и вышего образования Российской Федерации**

**Федеральное государственное бюджетное образовательное учреждение высшего образования**

**« Московский педагогический государственный университет »**

# ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №2

**Выполнил студент группы ЗОМ34-ПВШ261:** **Золотухин Андрей Александрович**

Принял: Жданов Сергей Александрович

Дата сдачи: 25.09.26

**Москва**

**2026**

# СОДЕРЖАНИЕ

1. Запуск примеров ..... 3
2. Задача 1 ..... 6
3. Задача 2 ..... 7

# 1. Запуск примеров

```
// Ознакомлен: Студент группы ЗОМ34-ПВШ261 Золотухин Андрей

// aray2D.cpp Программа C++ для иллюстрации двумерного массива.
// author: user
#include <iostream>
#include <string>
using namespace std;

int main(){
  // Определить 2D массив целых чисел (марицу 4x4)
  const int dim=4;
  int arr[dim][dim];
  /*
   Объявили 2D-массив с 4 строками (row) и 4 столбцами (column),
   после чего инициализировали массив значением (i+j) на каждой итерации
цикла.
   Затем печатаем 2D-массив, используя вложенный цикл, и в приведенном ниже
   выводе мы видим, что имеется 4 строки и 4 столбца.
  */
  cout <<" 2D array (matrix) " << dim <<"x" << dim<<":"<<endl;
  // Задать 2D массив, используя цикл for
  for (int i = 0; i < 4; i++) {
    for (int j = 0; j < 4; j++) {
      arr[i][j] = i + j;
    }
  }

  // Вывести элементы 2D массив в виде таблицы (матрицы)
   for (int i = 0; i < dim; i++) {
    for (int j = 0; j < dim; j++) {
      cout << " "  << arr[i][j] ;
    }
    cout << endl;
  }
  // вывод главной диагонали матрицы
  cout << " Elements of the main diagonal:" << endl;
  string s=" ";
  for (int i=0;i<dim;i++){
    cout <<s <<arr[i][i] <<endl;
    s+="  "; // добавить 2 символа пробела в строку s
  }
  // вывод побочной диагонали матрицы
  cout <<" Elements of secondary diagonal:"<<endl;
  int k=dim;
  string s1="       ";
```

```
  for (int j=0;j<dim;j++){
      cout << s1 << arr[j][k-1]<<endl;
      k--;
      s1.erase(1,2);//удалить 2 символа проболе из строки s1
  }
  int indxr=2; // индекс строки матрицы
  // вывод строки матрицы с индексом indxr
  cout <<" Vector-row of arr["<<indxr<<"][j] (j=0,1,2,3):"<<endl;
  for(int j=0;j<dim;j++)
    cout << " " <<arr[indxr][j];
  cout << endl;

  int indxc=1; //исндех столбца матрицы
  cout <<" Vector-column of arr[i]["<<indxc<<"] (i=0,1,2,3):"<<endl;
  for(int i=0;i<dim;i++)
    cout << " " <<arr[i][indxc]<<endl;
  cout << endl;
  return 0;
}
```

```
 2D array (matrix) 4x4:
 0 1 2 3
 1 2 3 4
 2 3 4 5
 3 4 5 6
 Elements of the main diagonal:
 0
   2
     4
       6
 Elements of secondary diagonal:
       3
     3
   3
 3
 Vector-row of arr[2][j] (j=0,1,2,3):
 2 3 4 5
 Vector-column of arr[i][1] (i=0,1,2,3):
 1
 2
 3
 4
```

```cpp
// Ознакомлен: Студент группы ЗОМ34-ПВШ261 Золотухин Андрей

// stru_example.cpp
// author: user
// C++ программа демонстрирует использование
// структуры данных struct, описывающая точку
// координатной плоскости
#include <iostream>
#include <cmath>

using namespace std;
// точка на плоскости
struct Point {
 double x, y; // x и  y координаты точки плоскости
};
// Некоторые функции декартовой геометрии на плоскости
// функция расстояния между двумя точками
double dist(Point pA, Point pB){
  return sqrt((pB.x - pA.x)*(pB.x - pA.x)+(pB.y - pA.y)*(pB.y - pA.y));
}
// Евклидова норма вектора из точки O(0,0) до точки (x,y)
double norm(Point A){
  return sqrt(A.x*A.x+A.y*A.y);
}
int main(){
  // тестирование структуры данных Point и функций на простом примере
  Point p1 = {1.0, 5.0};//координаты точки p1
  Point p2 = {2.5, 1.5};//координаты точки p2
  cout << " Point p1:  x = " << p1.x << ", y = " << p1.y <<endl;
  cout << " Point p2 (" << p2.x << ", " << p2.y <<")"<<endl;
  cout << " distance(p1,p2)= "<<dist(p1,p2)<< endl;
  cout << " norm(p1)="<<norm(p1)<<endl;// норма вектора (0,p1)
  cout<<endl;

  return 0;
}
```

```
 Point p1:  x = 1, y = 5
 Point p2 (2.5, 1.5)
 distance(p1,p2)= 3.80789
 norm(p1)=5.09902
```

# 2. Задача 1

```cpp
// Автор: Студент группы ЗОМ34-ПВШ261 Золотухин Андрей
// СДАП Лабораторная работа 2, Задача 1
// Язык: C++

#include <iostream>
#include <cmath>
#include <iomanip>

int main() {
    // Определить числа с плавающей точкой x, y, z
    double x = 2.7; // z = 2,7
    double y = 0.5; // y = 1/2
    double z = 1.0; // z = 1
    /* Объявил 3 переменных
     * и инициализировал их значениями из примера задачи.
     * Затем вычисляю a и b по формуле, данной в задаче
     * и печатаю их на экран.
     */
    // a = (sqrt(|x-1|) - cbrt(|y|)) / (1 + x²/2 + y⁴/4)
    double a = (std::sqrt(std::abs(x - 1.0)) - std::cbrt(std::abs(y))) / (1.0
+ (x * x) / 2.0 + (y * y * y * y) / 4.0);

    // b = x * (arctan(z) + e^(-(x+3)))
    double b = x * (std::atan(z) + std::exp(-(x + 3.0)));

    std::cout << "x = " << x << ", y = " << y << ", z = " << z << std::endl;
    std::cout << std::fixed << std::setprecision(16);
    std::cout << "a = " << a << std::endl;
    std::cout << std::fixed << std::setprecision(15);
    std::cout << "b = " << b << std::endl;
    // Ожидаемые значения: a = 0.1094574129127381; b = 2.129609147908283

    return 0;
}
```

```
x = 2.7, y = 0.5, z = 1
a = 0.1094574129127381
b = 2.129609147908283
```

# 3. Задача 2

```cpp
// Автор: Студент группы ЗОМ34-ПВШ261 Золотухин Андрей
// СДАП Лабораторная работа 2, Задача 2
// Язык: C++

#include <iostream>
#include <cmath>
#include <iomanip>

// Структура данных Point (точка на плоскости)
struct Point {
  double x, y;

  // Конструктор
  Point(
    double x_temp = 0.0,
    double y_temp = 0.0):
    x(x_temp),
    y(y_temp)
  {}

  // Евклидово расстояние до другой точки
  double distanceTo(const Point& anotherPoint) const {
      double distanceX = x - anotherPoint.x;
      double distanceY = y - anotherPoint.y;
      return std::sqrt(distanceX * distanceX + distanceY * distanceY);
  }
};

int main() {
  // Определяю структуры точки для каждой координаты
  Point A = {1.0, 2.0};
  Point B = {3.0, 6.0};
  Point C = {7.0, 1.0};
  /* Объявил структуру Point для 3 точек треугольника и инициализировал двумя
координатами для каждой.
   * Объявил длины сторон треугольника в отдельные переменные и
инициализировал их значениями, полученными из метода, которые находит
расстояние от одной точки до другой.
   * Объявил переменную для хранения значения площади и инициализировал её
значением, полученным через формулу векторного произведения пополам.
   * Затем все полученные значения и истинные значения из условия задачи.
   */

  // Длины сторон
  double ab = A.distanceTo(B);  // f
```

```
  double bc = B.distanceTo(C);  // g
  double ca = C.distanceTo(A);  // h

  // Площадь через векторное произведение (половина модуля)
  double area = std::fabs((B.x - A.x) * (C.y - A.y) - (B.y - A.y) * (C.x -
A.x)) / 2.0;

  std::cout << std::fixed << std::setprecision(2);
  std::cout << "Треугольник ABC:" << std::endl;
  std::cout << "  A = (" << A.x << ", " << A.y << ")" << std::endl;
  std::cout << "  B = (" << B.x << ", " << B.y << ")" << std::endl;
  std::cout << "  C = (" << C.x << ", " << C.y << ")" << std::endl;
  std::cout << std::endl;
  std::cout << "Сторона AB (f) = " << ab << std::endl;
  std::cout << "Сторона BC (g) = " << bc << std::endl;
  std::cout << "Сторона CA (h) = " << ca << std::endl;
  std::cout << std::setprecision(0);
  std::cout << "Площадь ABC    = " << area << std::endl;
  // Контроль: f=4.47, g=6.40, h=6.08, площадь=13

  return 0;
}
```

```
Треугольник ABC:
  A = (1.00, 2.00)
  B = (3.00, 6.00)
  C = (7.00, 1.00)

Сторона AB (f) = 4.47
Сторона BC (g) = 6.40
Сторона CA (h) = 6.08
Площадь ABC    = 13
```